

Simulación de camino de datos segmentado en MIPS32

Práctica 2

Nombre y apellidos: David Díaz Gómez-Escalonilla

DNI: 09845280-S

Índice

Índice de figuras **3**

1. Introducción **5**

2. Núcleo de la propuesta **7**
2.1. Diseño de la propuesta 7
2.2. Implementación 9
2.2.1. Memorias 10
2.2.1.1. Memoria de Instrucciones 10
2.2.1.2. Memoria de Datos 11
2.2.1.3. Banco de registros 11
2.2.2. Unidad aritmético-lógica (ALU) 12
2.2.3. Registros de acoplamiento 12
2.2.4. Segmentación 13
2.2.4.1. Funcionamiento de etapas 13
2.2.4.2. Unidad de Control 15
2.2.5. Resolución de riesgos 16
2.2.5.1. Forwarding Unit 17
2.2.5.2. Hazard Detection Unit 18
2.2.5.3. Flush, Stalls y burbujas 18
2.3. Simulación y salida 19
2.3.1. Representación de los ciclos 19
2.3.2. Programa principal de prueba 21
2.3.3. Casos específicos 22
2.3.3.1. RAW convencional 22
2.3.3.2. RAW simultáneo desde EX y MEM 23
2.3.3.3. RAW con load desde EX 24
2.3.3.4. Riesgo de control tras ejecución de salto 24

3. Conclusiones **27**

Bibliografía **29**

Índice de figuras

1. Ejecución uniciclo no segmentada frente a ejecución segmentada. Fuente: (Patterson & Hennessy, 2000, fig. 6.3, p. 429). . . . 6
2. Arquitectura general del simulador . . . 8
3. Composición del cauce segmentado . . . 8
4. Camino de datos segmentado . . . 9
5. Implementación de las memorias. . . . 10
6. Implementación del banco de registros. . . . 11
7. Implementación de la clase ALU. . . . 12
8. Coordinación de las etapas de segmentación . . . 14
9. Atributos de ControlUnit. . . . 16
10. Lógica de llamadas de ForwardingUnit. . . . 17
11. Representación de ciclo . . . 20
12. Programa principal de prueba en ensamblador MIPS32. . . . 21
13. Resultados de la ejecución . . . 22
14. RAW convencional . . . 23
15. RAW simultáneo desde EX y MEM . . . 23
16. RAW con load desde EX . . . 24
17. Riesgo de control tras ejecución de salto . . . 25

1. Introducción

En un camino de datos se cargan y manipulan distintos datos procedentes de memoria o del banco de registros con el objetivo de procesarlos y permitir al procesador la ejecución de instrucciones que conforman una tarea. Todo esto forma parte de la unidad de procesamiento, pero para poder interpretar todas estas instrucciones correctamente se precisa de una unidad de control, encargada de dirigir y coordinar el procesamiento. Ambas unidades son necesarias para el correcto funcionamiento de un procesador, y en esta memoria nos centraremos en los procesadores cuya arquitectura corresponde a la MIPS32, una arquitectura de conjunto de instrucciones reducido (RISC) de 32 bits diseñada para ejecutar aplicaciones de alto rendimiento en una amplia variedad de sistemas.

La segmentación o pipelining es una técnica de implementación que busca aumentar las prestaciones de un computador dividiendo la ejecución de cada instrucción en varias etapas y solapando la ejecución de distintas instrucciones. Esta técnica interesa aplicarla en una gran mayoría de casos, ya que aumenta notablemente la productividad del procesador debido a que se ejecuta un número mayor de instrucciones por segundo, pese a que no mejora la latencia de cada instrucción, tal y como podemos observar en la Figura 1, donde cada instrucción uniciclo no segmentada tiene una latencia de 8 ns frente a los 10 ns de latencia de las instrucciones segmentadas.

Para poder ejecutar instrucciones aplicando segmentación es preciso dividir cada instrucción en etapas:

- IF (Instruction Fetch): Se carga la instrucción desde la memoria de instrucciones.
- ID (Instruction Decode): Decodificación de la instrucción y lectura del banco de registros.
- EX (Execution): Ejecución de operaciones aritmético-lógicas y cálculo de direcciones.
- MEM (Memory Access): Lectura o escritura en la memoria de datos.
- WB (Write Back): Escritura del resultado en el banco de registros.

Para aplicar correctamente segmentación, las distintas instrucciones que estén en el camino de datos deben avanzar por el cauce de forma solapada, almacenando los datos de cada instrucción en registros de acoplamiento situados entre cada fase. Sin embargo, se pueden dar situaciones de riesgos (hazards) que impiden que una instrucción pueda continuar normalmente su ejecución en el siguiente ciclo. En un procesador que aplica segmentación aparecen principalmente riesgos de datos cuando una instrucción requiere de un dato que está siendo manipulado por una instrucción previa cuya ejecución aún no ha terminado. Existen distintos tipos de riesgos de datos, pero el más relevante es RAW (Read After Write), donde una instrucción trata de leer un registro cuyo valor aún no ha sido actualizado por una instrucción anterior. Debido a esto es necesario aplicar distintos mecanismos como el bloqueo del cauce, reorganización de instrucciones en el programa en ensamblador o anticipación (forwarding). En este último mecanismo, el forwarding, se envía directamente el dato requerido a la instrucción que precisa del mismo sin tener que esperar a que se escriba en el banco de registros.

También se pueden presentar riesgos de control, donde entran al cauce instrucciones que no corresponden al flujo de ejecución. Esto se da en saltos y bifurcaciones, donde distintas instrucciones entran al cauce cuando todavía se está calculando el nuevo PC o determinando si se realiza el salto. Para tratar estos riesgos se puede: detener el cauce (stall), reducir el retardo del salto, predecir el salto o añadir una ranura de retardo (delay slot).

El objetivo de esta práctica es implementar una simulación en Python de un procesador MIPS32 segmentado, donde se simule la ejecución y fraccionamiento de una instrucción en las cinco etapas de segmentación correspondientes, pudiendo solapar hasta cinco instrucciones en el cauce, cada una en una etapa distinta. Además, la implementación debe ser fiel al hardware que compone

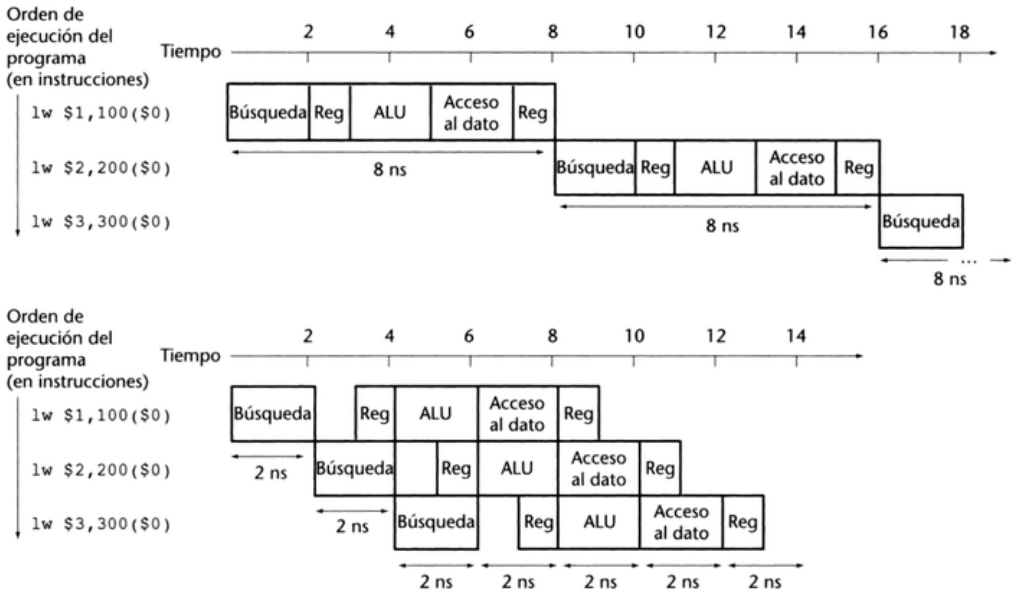


Figura 1: Ejecución uniciclo no segmentada frente a ejecución segmentada. Fuente: (Patterson & Hennessy, 2000, fig. 6.3, p. 429).

la arquitectura, teniendo clases de objetos que correspondan a distintos componentes como el banco de registros, las memorias, la unidad aritmético-lógica, o los registros de acoplamiento. También se debe mostrar el flujo de ejecución de manera fácil e intuitiva, y la simulación debe cubrir al menos ocho instrucciones aritmético-lógicas, cuatro instrucciones de escritura y lectura en memoria y tres saltos. Además, se deben implementar mecanismos que permitan resolver los riesgos de datos y de control del procesador.

A continuación, se presenta el diseño e implementación de esta simulación, donde se muestra el conjunto de clases que compone el programa, además de la relación entre distintas clases y las funcionalidades que cubren, seguido de los diagramas de clases UML correspondientes. Posteriormente se explica la implementación de cada clase a partir de su código, con sus atributos y sus métodos, acompañado de anotaciones relacionadas con las decisiones tomadas respecto al diseño individual de cada clase. En la sección siguiente se muestran los resultados de la ejecución de nuestro programa en ensamblador propuesto antes de la implementación de la práctica, donde se comprobarán los resultados obtenidos con los esperados, además de analizar casos más concretos de la ejecución. Finalmente, se exponen las conclusiones extraídas de la realización de la práctica.

2. Núcleo de la propuesta

Se ha desarrollado un simulador de un procesador MIPS32 segmentado mediante Python, aplicando programación orientada a objetos. Los principales componentes del camino de datos se han implementado mediante una clase, definiendo sus atributos y métodos en función de las distintas responsabilidades de cada uno. Para coordinar la ejecución se ha creado una clase CPU, siendo la principal responsable de la coordinación de la ejecución, y contiene las instancias de la Unidad de Procesamiento y de la Unidad de Control, siendo las principales que contienen el hardware y las señales de control respectivamente. Respecto a los distintos riesgos o hazards, se ha implementado un Forwarding Unit, responsable de detectar los distintos RAW que se puedan solucionar mediante anticipación. Sin embargo, Forwarding Unit no puede cubrir todos los hazards, por lo que también se ha implementado un Hazard Detection Unit para detectar los riesgos relacionados con instrucciones load (lw), habilitando que se apliquen stalls e inserten burbujas en el cauce para solucionar el riesgo.

A continuación, se explicará en profundidad el diseño, analizando la arquitectura general y la relación de sus distintas clases. Posteriormente, se explicará la implementación del programa, analizando individualmente cada una de estas clases, y finalmente se ejecutará una serie de programas para poner a prueba nuestra simulación.

2.1. Diseño de la propuesta

Como se ha explicado anteriormente, se ha aplicado un diseño orientado a objetos a la arquitectura del procesador, permitiendo separar responsabilidades en distintas clases. Cada una de estas clases representa principalmente un componente (Memoria, ALU, Banco de registros, PC, etc), aunque también hay clases encargadas de la transmisión de señales de control (ControlUnit), de la segmentación y sus registros de acoplamiento (Pipeline) y de la detección de riesgos (ForwardingUnit y HazardDetectionUnit).

En la Figura 2 observamos cómo la CPU contiene referencias de ProcessingUnit y ControlUnit. Esto es debido a que en esta clase se ejecuta el método `run()`, encargado de ejecutar todo el programa que permite realizar la simulación, por lo que tener acceso a los componentes de la unidad de procesamiento y a las señales de control de la unidad de control facilita mucho la implementación y coordinación del programa. El ProcessingUnit representa la unidad de procesamiento, y es la clase responsable de ejecutar cada una de las fases de la segmentación. Para poder realizar eficientemente esta tarea es necesario tener acceso a los componentes del camino de datos, por lo que se almacenan como atributos en esta clase. Durante la ejecución de cada una de las fases de segmentación se realiza el acceso a los componentes correspondientes, cuyo comportamiento dependerá de las señales de control de cada instrucción. También se almacena una instancia de Pipeline, donde se guardan los distintos registros de acoplamiento, permitiendo almacenar la información correspondiente a cada instrucción en cada etapa de segmentación. Respecto al `ControlUnit`, representa la unidad de control, y se encarga de almacenar y transmitir las señales de control de cada instrucción, generadas en la fase de decodificación. Su instancia es pasada como parámetro a la hora de ejecutar cada fase de segmentación, tarea facilitada gracias a la clase de CPU.

ProcessingUnit contiene las instancias de las clases de los componentes, lo que permite el acceso a los mismos en cada fase de ejecución, teniendo un método por cada fase de segmentación, desde donde se llaman a los distintos componentes, como el banco de registros en instruction decode o la unidad aritmético-lógica en execution. Los atributos de esta clase son: ProgramCounter, RegisterFile, ALU, Instruction Memory, Data Memory y la clase Pipeline para agrupar los registros de acoplamiento. Cada una de estas clases es instanciada al inicio de la ejecución y se almacenan en la instancia de la unidad de procesamiento. Posteriormente se emplean en distintas fases mediante los métodos de cada uno de estos componentes. Respecto a las memorias, tanto Instruction Memory como Data Memory heredan de la clase Memory, que contiene como atributo

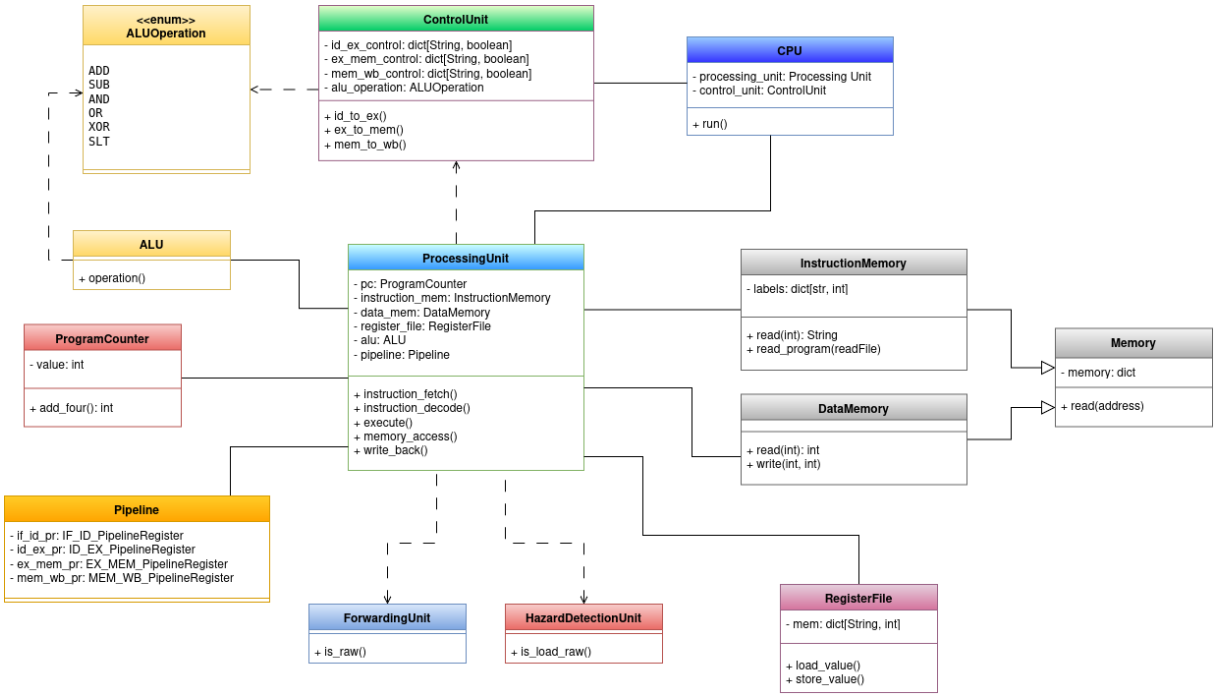


Figura 2: Diagrama UML de clases de la arquitectura general del simulador.

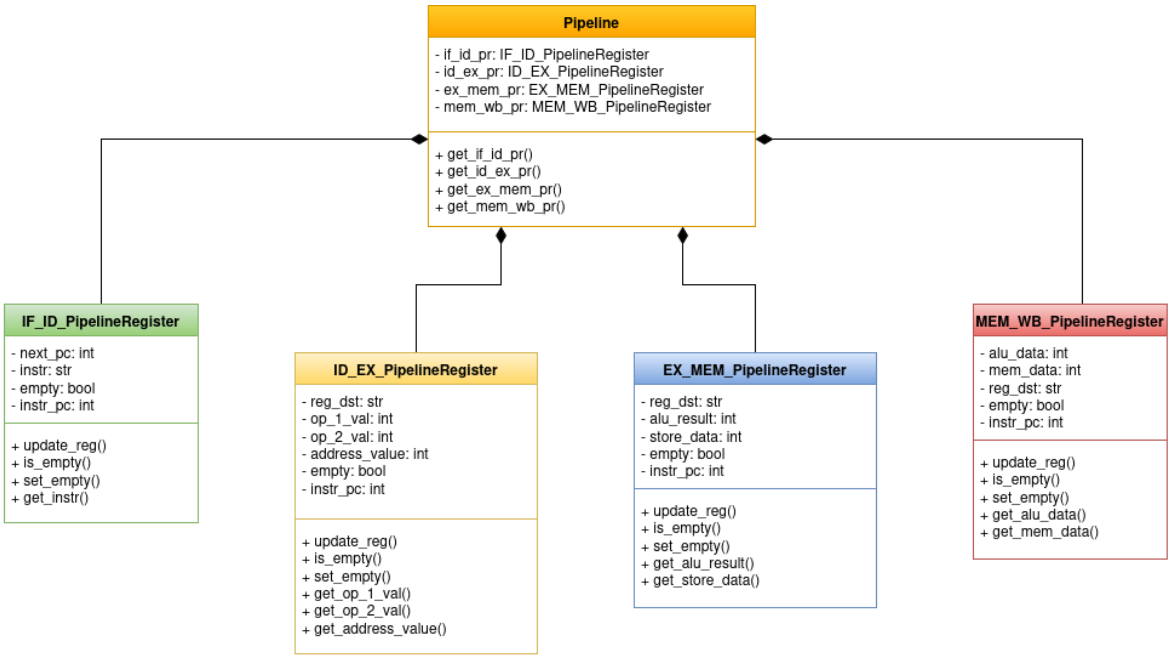


Figura 3: Diagrama UML de clases de la composición del cauce segmentado.

un diccionario vacío y el método read() para su lectura. También existe una enumeración que se puede apreciar en la Figura 2, siendo ALUOperation, de tipo enum. Contiene una enumeración de todas las posibles operaciones que puede realizar la ALU, y su valor se establece en la fase de decodificación, y se transmite mediante la unidad de control ControlUnit.

Respecto a las clases ForwardingUnit y HazardDetectionUnit, se establece una relación de dependencia en la Figura 2. Esto es debido a que ProcessingUnit requiere de los métodos estáticos implementados en estas clases para la detección de RAW, pero no es necesario almacenar una instancia de cada clase como atributo del ProcessingUnit.

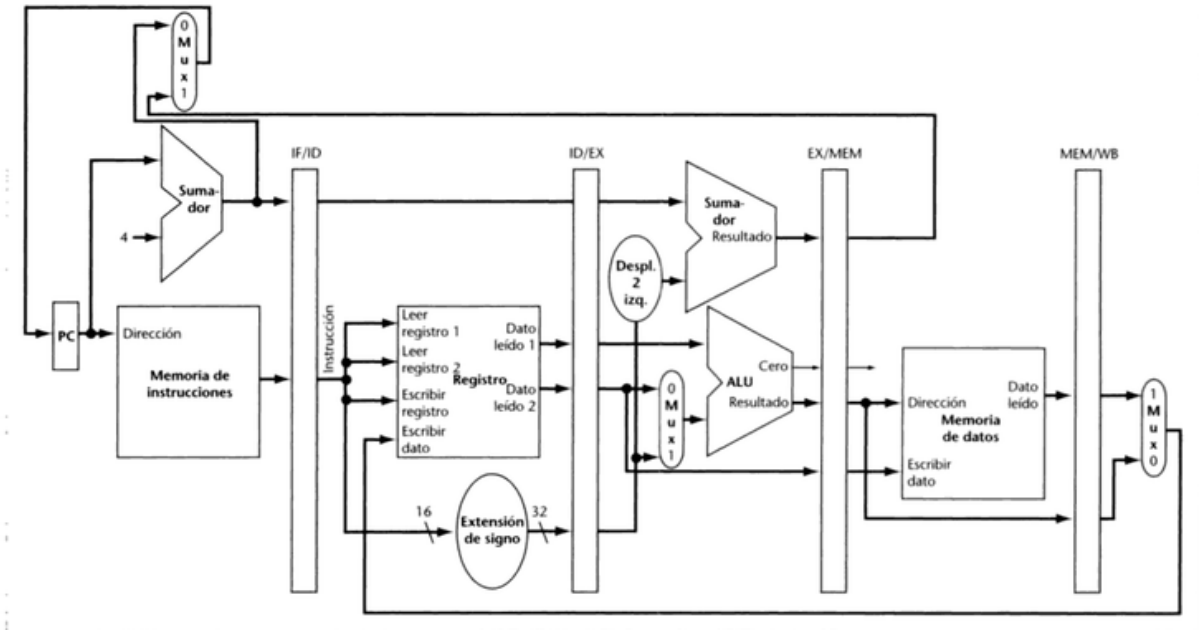


Figura 4: Versión segmentada del camino de datos. Fuente: (Patterson & Hennessy, 2000, fig. 6.12, p. 442).

En el diagrama de la Figura 3 se representa la clase Pipeline junto con los registros de acoplamiento que la conforman, los cuales se instancian en el constructor del propio Pipeline, lo que justifica la relación de composición de los registros de acoplamiento respecto a Pipeline. Estos registros de acoplamiento se emplean para almacenar la información de cada instrucción para poder ejecutarla en cada fase. La información guardada puede variar en función del registro de acoplamiento, debido a que en cada fase se consumen distintos datos, pero se generan otros nuevos, como son los operandos y el resultado de la ALU durante la fase de ejecución. Es importante matizar que el almacenamiento y transmisión de las señales de control es responsabilidad del ControlUnit y no de los registros de acoplamiento. En la Figura 4 observamos que, debido a que trabajamos con cinco fases de segmentación, empleamos cuatro registros de acoplamiento, siendo:

- IF/ID: Almacena la instrucción cargada desde la memoria de instrucciones mediante su PC.
- ID/EX: Almacena los operandos, el inmediato y el registro destino.
- EX/MEM: Almacena el resultado de la ALU, el posible dato a almacenar en la memoria de datos y propaga el registro destino.
- MEM/WB: Contiene el resultado de la ALU en caso de no acceder a memoria o el posible dato cargado desde memoria de datos, siendo uno de ellos almacenado en el registro destino.

También se emplean atributos `empty` e `instr_pc` para indicar cuándo un registro de acoplamiento está vacío y representar burbujas, y para poder representar la instrucción a partir de su dirección en terminal, respectivamente.

2.2. Implementación

En esta sección se describe la implementación de cada una de las clases del programa, explicando su código, con sus correspondientes atributos y métodos, justificando cada una de las razones de diseño tomadas durante el proceso de implementación.

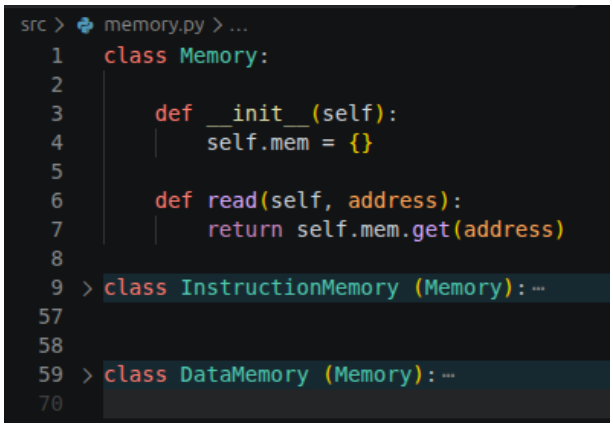


Figura 5: Implementación de las memorias.

2.2.1. Memorias

En la Figura 5, se ha creado una clase padre Memory de la que heredan la InstructionMemory y DataMemory. Se proporciona un atributo mem, siendo un diccionario que se inicializa vacío, y que busca simular el comportamiento de ambas memorias, ya que se introduce una key de tipo Integer a modo de dirección de memoria, y se devuelve un value, cuyo tipo varía en función de la memoria que lo implemente, ya que en InstructionMemory se devuelven instrucciones mediante Strings y DataMemory devuelve datos en forma de Integers. Para poder acceder a este diccionario `mem` se proporciona el método `read(address)`, que recibe el Integer address y se devuelve el valor correspondiente.

2.2.1.1 Memoria de Instrucciones

Clase responsable de procesar y guardar cada una de las instrucciones del programa en ensamblador, además de proporcionar métodos que permitan acceso a la instrucción correspondiente a su PC. Debido a que hereda de la clase Memory, hereda el diccionario mem como atributo, donde se almacenará cada PC(key) con su correspondiente instrucción (value). También hereda el método `read()` para extraer las instrucciones del diccionario. Para poder implementar la memoria de instrucciones de manera óptima, se le añade el atributo `labels`, siendo un diccionario que almacena cada etiqueta del programa con su dirección correspondiente, y los siguientes métodos:

- `read_program(program_file)`: Se encarga de realizar la lectura del archivo .txt con el programa en ensamblador donde se procesa cada línea del programa diferenciando entre instrucciones y etiquetas, además de normalizarlas en el caso de que hubiera comentarios, espacios en blanco o tabulaciones. Cada instrucción es almacenada asignando un PC múltiplo de cuatro, y cada etiqueta es guardada en el diccionario labels con su PC correspondiente, que se empleará más adelante en el método privado `_replace_labels()`, que se llama al final del método.
- `_replace_labels()`: Método privado responsable de sustituir cada etiqueta o label que va incluida en cada instrucción de salto o bifurcación por la diferencia entre el PC de la instrucción y de la etiqueta destino.
- `print_mem()`: Muestra todos los keys con sus values correspondientes del atributo mem, mostrando en este caso cada PC con su instrucción correspondiente.
- `is_out_of_bounds(address)`: Método empleado en la fase de instruction fetch, donde se comprueba que el PC leído no supera la última dirección de la memoria de instrucciones.
- `_is_instruction(line)`: Método privado empleado para reconocer cada instrucción durante el procesamiento del fichero .txt con el programa en ensamblador.



Figura 6: Implementación del banco de registros.

2.2.1.2 Memoria de Datos

Al igual que InstructionMemory, hereda el atributo `mem`, inicialmente vacía, y el método `read()` para acceder al diccionario que simula la memoria de datos. En este caso, el value almacenado en el diccionario no es un String, sino un entero. Además se añaden los siguientes métodos:

- `write(address, value)`: Método que permite realizar operaciones de escritura, llamado desde la fase de segmentación MEM en el caso de que se ejecute una instrucción store. Se proporciona como parámetros la dirección de escritura y el valor a almacenar, sobrescribiendo el anterior, o creando un nuevo key-value en el diccionario en el caso de que el address proporcionado no tuviera un valor asignado anteriormente.
- `print_data_mem()`: Empleado para mostrar las direcciones de memoria de datos con sus valores almacenados correspondientes. Se emplea principalmente para hacer seguimiento del programa mediante terminal.

2.2.1.3 Banco de registros

La clase RegisterFile representa el banco de registros de nuestro camino de datos, y al igual que en la clase Memory, tiene un atributo `mem`, el cual es un diccionario que simula el acceso al banco de registros empleando los propios registros como key. Como se observa en la Figura 6, a diferencia de las memorias de datos e instrucciones, el banco de registros se inicializa con cada uno de los registros de MIPS32 con el valor None, a excepción del registro `$zero` y el registro stack pointer `$sp`, que se inicializan con los enteros 0 y 512 (múltiplo de 4 arbitrario) respectivamente. Se incluyen los siguientes métodos:

- `store_value(k, v)`: Almacena en el registro pasado como key el value pasado como parámetro, siempre que el registro pertenezca al conjunto de registros de MIPS32, y no sea el registro `$zero`, esto último es debido a que MIPS32 no permite la escritura de este registro.
- `load_value(k)`: Carga el valor guardado en el registro pasado como parámetro.
- `print_register_file()`: Muestra cada registro del banco de registros con su valor almacenado, en el caso de que tenga uno asignado.

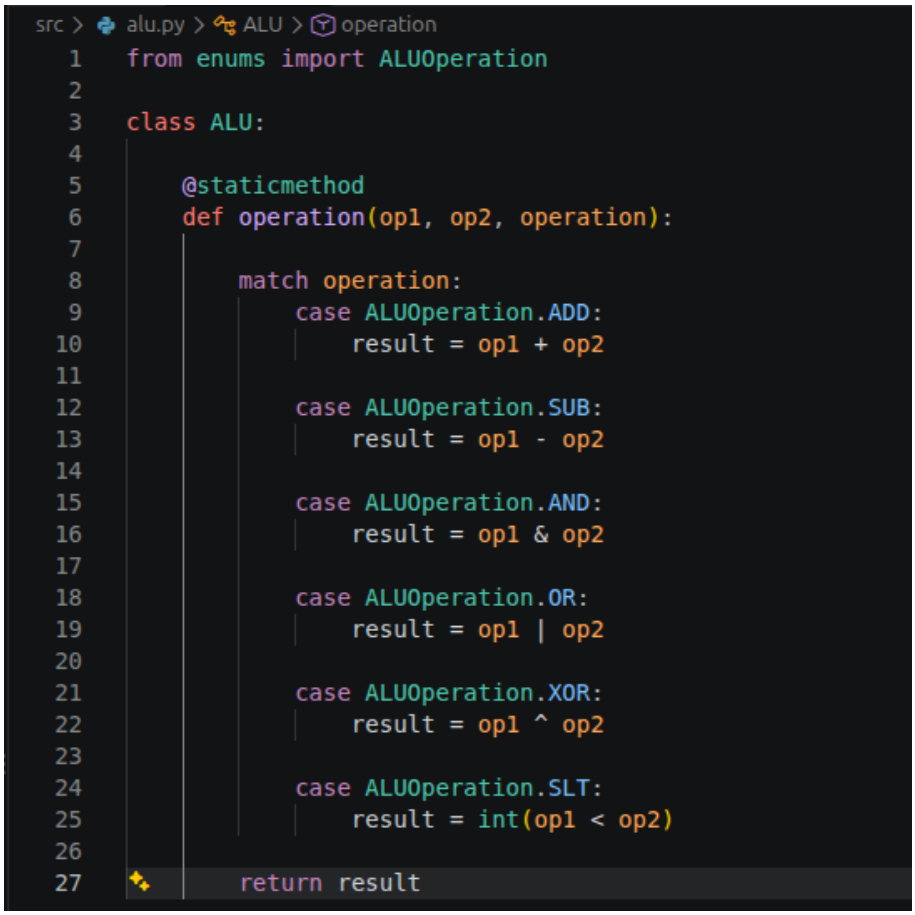


Figura 7: Implementación de la clase ALU.

2.2.2. Unidad aritmético-lógica (ALU)

Para implementar la ALU se ha creado una clase ALU, que contiene un único método estático, y un enum ALUOperation, que enumera todas las operaciones que puede realizar la ALU. Respecto a la clase ALU, no contiene ningún atributo, ya que se considera más óptimo que toda la información requerida que se pase como parámetro en las funciones. Respecto a las funciones, solo se ha requerido la implementación de un método: `operation(op1, op2, operation)`, donde recibe los dos operandos correspondientes, y la operation, de tipo ALUOperation, que indica cómo se van a emplear ambos operandos mediante el match-case de la Figura 7. El ALUOperation puede realizar operaciones que requieran de: ADD, SUB, AND, OR, XOR y SLT. Respecto a la señal Zero, se ha decidido que la ALU no devuelva ninguna señal lógica, y que simplemente, en el método correspondiente a la fase EX, se compruebe si el resultado de la ALU es igual a cero.

2.2.3. Registros de acoplamiento

Como se muestra en el diagrama de la Figura 3, se han implementado cuatro registros de acoplamiento, y todos ellos se instancian en el constructor de la clase Pipeline, guardándose como atributos. Gracias a esta composición, y a la implementación de los gets() y sets() correspondientes a cada registro de acoplamiento, tenemos acceso a todo el pipeline siempre que tengamos una instancia de la clase Pipeline. Cada registro tiene sus propios atributos en función de qué intersección de fases representan, aunque todos ellos tienen un atributo booleano `empty`, el cual se emplea para indicar que el registro está vacío, en caso de que no se haya ejecutado ninguna instrucción en la etapa correspondiente. También se emplea para indicar burbujas en el cauce causadas por flushs o stalls. Cada registro de acoplamiento tiene sus propios métodos, pero hay

dos que son comunes en todos:

- `update_reg()`: Actualiza el registro de acoplamiento al que pertenece el método. Debido a que cada registro almacena datos distintos de cada instrucción, los parámetros de la función cambian en función de la etapa de segmentación. Se ejecutan al final de cada método correspondiente a cada etapa del pipelining, actualizando cada registro con sus datos correspondientes.

- `is_empty()`: Método que permite comprobar si un registro de acoplamiento tiene información útil o no, previniendo que se ejecute una instrucción innecesariamente.

2.2.4. Segmentación

Como se ha mencionado, tenemos cuatro registros de acoplamiento, cuyos valores se actualizan al ejecutar cada método correspondiente a cada etapa de segmentación. Estos métodos pertenecen a la clase ProcessingUnit, pero se llaman desde el método `run()` de la clase CPU mediante la instancia de la unidad de procesamiento, almacenada como atributo de CPU. Sin embargo, también se utilizan funciones del ProcessingUnit y del ControlUnit para facilitar la coordinación, y cuyas instancias están almacenadas como atributos en la clase CPU.

2.2.4.1 Funcionamiento de etapas

El método `run()` contiene toda la ejecución dentro de un bucle while, cuya condición para mantener su ejecución es que el método `processing_unit.is_finished()` devuelva false, ya que es una función del ProcessingUnit donde se comprueba que todos los registros de acoplamiento estén vacíos llamando al método `is_empty()` en cada uno de ellos, además de comprobar que el PC haya superado la última dirección del programa.

El orden de ejecución de las fases de segmentación implementado en el código de la Figura 8 es el siguiente: WB → EX(sin cargar resultados en EX/MEM) → MEM → Cargar resultados de EX en EX/MEM → ID → IF. Durante la ejecución seguimos este orden principalmente para la ejecución secuencial de las etapas evitando sobrescritura entre registros y para que EX calcule sus resultados antes que MEM pero sin actualizar el registro EX/MEM. Para que EX pueda ejecutarse antes que MEM sin realizar sobrescritura, primero se ejecuta el método correspondiente a EX, pero los resultados no se cargan directamente en EX/MEM, sino que se almacenan en un diccionario que contiene cada uno de los posibles valores, los cuales se cargan tras la ejecución de la fase MEM.

Para la implementación de los métodos, se ha pasado como parámetro la instancia de la unidad de control a los métodos cuyas etapas de segmentación consumen señales de control, generadas en la fase ID. Esta instancia se emplea para poder comprobar las señales de control de cada fase. Los métodos correspondientes a las fases de la segmentación son los siguientes:

- `write_back(control_unit)`: Extrae el dato a escribir mediante la señal de control MemtoReg, y lo almacena en el banco de registros si la señal RegWrite está activa.

- `execute(control_unit)`: Su ejecución se particiona en tres posibles caminos en función de las señales de control Jump y Branch, en el caso de que ninguna esté activa se trata la instrucción como una operación aritmético-lógica, ya sea con o sin inmediato. En el caso de que tengamos una operación de salto o bifurcación, se calculan las direcciones de salto sumándole al PC de la instrucción el offset correspondiente. El método devuelve una tupla de dos valores, donde el primero corresponde al diccionario que contiene los distintos elementos que devuelve la fase EX (resultado de la ALU, dato del store, registro destino, etc) para ser almacenados en EX/MEM posteriormente, y después se devuelve la variable jump_address, cuyo valor es la dirección de salto en el caso de que la instrucción sea Jump o sea un Branch o BranchN (branch not equal) y se cumpla la condición de salto. En caso contrario, se devuelve un None.

```
while not self.processing_unit.is_finished():

    cycle+=1
    stall_signal = False

    # WB
    self.processing_unit.write_back(self.control_unit)

    # EX execution without store
    (ex_mem_copy, jump_address) = self.processing_unit.execute(self.control_unit)

    # MEM
    result_mem, reg_dst_mem = self.processing_unit.memory_access(self.control_unit)
    if not self.processing_unit.get_pipeline().get_mem_wb_pr().is_empty():
        self.control_unit.mem_to_wb()
    else:
        self.control_unit.flush_mem_wb()

    # EX upload Registers
    result_ex, reg_dst_ex = self.processing_unit.update_ex_mem_reg(ex_mem_copy)
    if not self.processing_unit.get_pipeline().get_ex_mem_pr().is_empty():
        self.control_unit.ex_to_mem()
    else:
        self.control_unit.flush_ex_mem()

    # ID
    if jump_address is not None:
        # flush instrs after active conditional jump
        self.processing_unit.get_pipeline().get_id_ex_pr().set_empty(True)
        self.control_unit.flush_id_ex()
    else:
        stall_signal = self.processing_unit.instruction_decode(self.control_unit,
                                                               result_ex,
                                                               reg_dst_ex,
                                                               result_mem,
                                                               reg_dst_mem)

    # IF
    pc_src = (jump_address is not None)

    self.processing_unit.instruction_fetch(pc_src, jump_address, stall_signal)
```

Figura 8: Implementación del método run() de la clase CPU, encargado de coordinar las etapas de segmentación.

- `memory_access(control_unit)`: En función de las señales de control MemWrite y MemRead extraídas de la instancia control_unit pasada como parámetro, se realiza una operación de escritura o lectura, o directamente se transmite el dato al registro MEM/WB para su posterior escritura en el banco de registros, sin tener que pasar por la memoria de datos, al igual que el registro destino, que se transmite directamente al registro de acoplamiento siguiente. Esta función devuelve una tupla con los valores correspondientes al dato que se va a escribir en el banco de registros, determinado por la señal MemToReg, y la dirección del registro destino. Esto se hace con el objetivo de pasar estos datos como parámetros a la fase ID, simulando el cortocircuito de datos gestionado Forwarding Unit desde la fase de decodificación.
- `update_ex_mem_reg(ex_mem_copy)`: Se llama inmediatamente después de MEM, y almacena los valores del diccionario que corresponden al registro EX/MEM. Al igual que `memory_access()`, la función también retorna una tupla con el resultado de la ALU y el registro destino, empleado por Forwarding Unit en la fase ID.

- `instruction_decode(control_unit, result_ex, reg_dst_ex, result_mem, reg_dst_mem)`: Siendo el método más extenso de la segmentación, es responsable de decodificar cada instrucción, identificando su tipo (load, store, bifurcación, aritmetico-lógica básica o con inmediato, o salto) y extrayendo los valores correspondientes (registros, operandos, inmediatos y desplazamientos) para cargarlos en el registro de acoplamiento ID/EX. Además, como se mencionó anteriormente, en esta fase se generan para cada tipo de instrucción las señales de control correspondientes. Toda esta tarea se realiza mediante un match-case, cuyo acceso a sus cases varía en función del código de operación extraído de la instrucción. En este método también se realiza la detección de riesgos o hazards, por lo que también se emplean los métodos estáticos de las clases HazardDetectionUnit y Forwarding Unit, y se implementa toda la lógica necesaria para su correcto funcionamiento, en la que profundizaremos posteriormente. Debido a que la detección de riesgos se realiza en el cuerpo de la función, en el caso de que se detecte un hazard respecto a un load, ID introduce una burbuja en ID/EX y devuelve como valor de retorno la señal stall. Este valor, declarado como booleano, se devuelve al método `run()` de CPU, desde donde se pasa como parámetro al método `instruction_fetch()` en caso de que haya un RAW respecto a un load para poder paralizar el cauce en las fases de IF e ID.

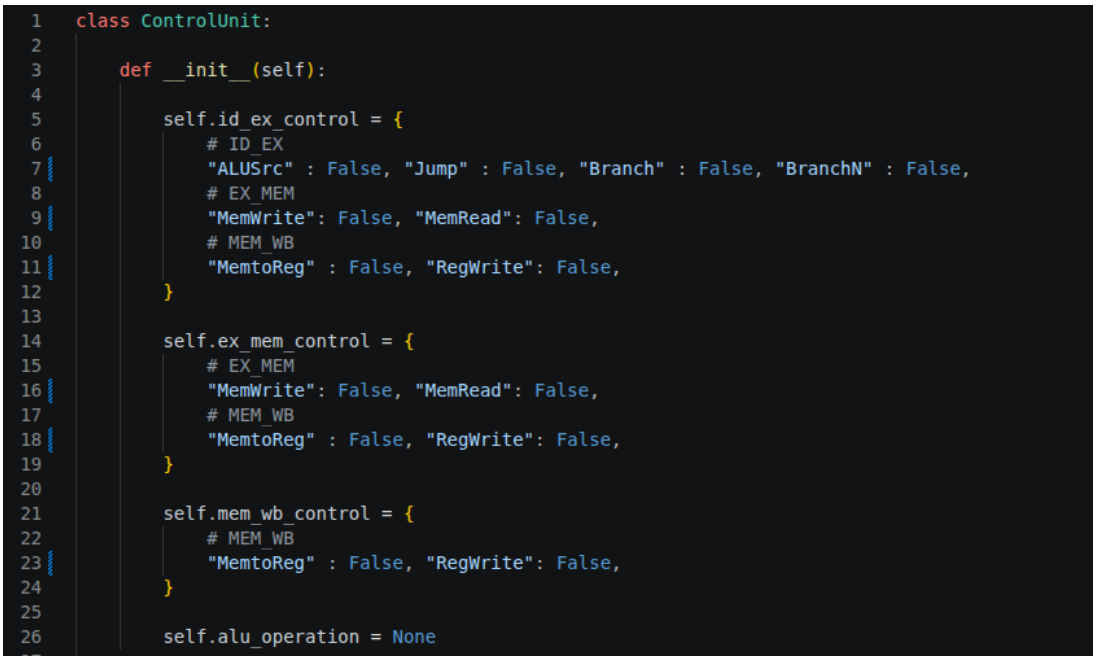
- `instruction_fetch(pc_src, jump_address, stall_signal)`: Método responsable de la carga de la instrucción correspondiente a la ejecución del programa en ensamblador, siempre que el parámetro stall_signal no indique lo contrario. La dirección de la instrucción se extrae en función del parámetro pc_src, ya que puede corresponder a la instancia almacenada como atributo del program counter o a la dirección de salto jump_address. En el caso de que no haya un stall y el PC contenga una dirección válida, al program counter siempre se le suma cuatro al final del método, actualizando el valor en nuestra instancia de PC. Respecto a que un PC tenga una dirección válida, dentro del método se comprueba que el PC cargado esté dentro de los límites del programa, asegurándose de que no sobrepasa la última dirección PC de la memoria de instrucciones mediante el método `is_out_of_bounds()`

2.2.4.2 Unidad de Control

La clase ControlUnit corresponde a la unidad de control, y está compuesta por tres diccionarios, donde cada uno contiene las señales de control de los registros de acoplamiento ID/EX, EX/MEM y MEM/WB, tal y como se observa en la Figura 9. Este conjunto de señales de control lo componen tanto las señales que se consumen en la etapa siguiente del registro de acoplamiento como las señales de las fases posteriores. Estas señales de control son las claves de los diccionarios, y el valor almacenado corresponde al booleano que indica si la señal está activa o no. Para esta simulación se ha descartado la señal RegDst, empleada originalmente para indicar cuál es el registro destino de las instrucciones que escriben en el banco de registros, debido a que se ha simplificado simplemente pasando desde la fase ID directamente el registro destino de la instrucción. También hay un cuarto atributo, el `alu_operation`, que almacena la operación a realizar en la ALU.

Las señales de control se generan en la fase de ID, en el método `instruction_decode()`, donde se aprovecha la decodificación y categorización de la instrucción para indicar cada una de las señales de control requeridas según el tipo al que pertenece, además de la ALUOperation que se pasará a la ALU posteriormente.

Las señales de cada instrucción se van transmitiendo entre fases a través de nuestra instancia de la unidad de control, `control_unit` pasada como parámetro a las fases siguientes de la decodificación. Esto se hace con el objetivo de respetar la distribución de tareas que hay entre la unidad de control y la unidad de procesamiento, ya que a ControlUnit le corresponde el tratamiento de las señales de control mientras que a ProcessingUnit los datos útiles de cada instrucción transmitidos por el cauce del camino de datos. Dentro de cada fase, se consumen las señales lógicas requeridas en esa etapa de segmentación, transmitiendo al diccionario correspondiente a la



```
class ControlUnit:

    def __init__(self):

        self.id_ex_control = {
            # ID_EX
            "ALUSrc" : False, "Jump" : False, "Branch" : False, "BranchN" : False,
            # EX_MEM
            "MemWrite": False, "MemRead": False,
            # MEM_WB
            "MemtoReg" : False, "RegWrite": False,
        }

        self.ex_mem_control = {
            # EX_MEM
            "MemWrite": False, "MemRead": False,
            # MEM_WB
            "MemtoReg" : False, "RegWrite": False,
        }

        self.mem_wb_control = {
            # MEM_WB
            "MemtoReg" : False, "RegWrite": False,
        }

        self.alu_operation = None
```

Figura 9: Atributos de la clase ControlUnit.

fase siguiente solo las señales de control necesarias para las fases posteriores. Para esta transmisión de señales, ControlUnit cuenta con tres métodos:

- `id_to_ex(control_signals, alu_operation)`: Transmite las señales de control generadas durante la fase ID, junto con el ALUOperation.
- `ex_to_mem()`: Transmite las señales correspondientes a MEM y WB, descartando las señales correspondientes EX.
- `mem_to_wb()`: Transmite las señales correspondientes a WB, descartando las señales correspondientes MEM.

La clase ControlUnit también cuenta con los métodos: `flush_id_ex()`, `flush_ex_mem()` y `flush_mem_wb()`. Cada método corresponde a un diccionario, y desactivan cada una de las señales de control poniéndolas en False, en caso de que haya una burbuja.

2.2.5. Resolución de riesgos

En nuestra arquitectura existen distintos riesgos o hazards que pueden alterar el flujo de ejecución del programa, habiendo dos tipos posibles de riesgos:

- Riesgos de datos RAW: Dado cuando se realiza la lectura de un dato que se ha actualizado previamente pero que aún no se ha actualizado en el banco de registros.
- Riesgos de control: Dado cuando se realiza un salto y entran al cauce del camino de datos instrucciones posteriores a la dirección de la instrucción que ejecuta el salto, las cuales no corresponden al flujo.

Para detectar los riesgos de datos se han implementado las clases ForwardingUnit y HazardDetectionUnit, además de distintos mecanismos de stall, burbuja y flush. Respecto a los riesgos de control, se tratan aplicando una lógica de salto en `run()` en CPU, identificando si se ha ejecutado un salto o no a partir de la variable `jump_address`.

```
# Forwarding EX
if ForwardingUnit.is_raw(op_1_reg, reg_dst_ex, control_unit.get_ex_mem_control()):
    op_1_val = result_ex
    raw_case_ex["addr_reg"] = op_1_reg
    raw_case_ex["operands"].append(1)

if ForwardingUnit.is_raw(op_2_reg, reg_dst_ex, control_unit.get_ex_mem_control()):
    op_2_val = result_ex
    raw_case_ex["addr_reg"] = op_2_reg
    raw_case_ex["operands"].append(2)

# Forwarding MEM
if not raw_case_mem["forwarding_disabled"]:
    if ForwardingUnit.is_raw(op_1_reg, reg_dst_mem, control_unit.get_mem_wb_control()):
        raw_case_mem["operands"].append(1)

        if 1 in raw_case_ex["operands"]:
            raw_case_mem["forwarding_disabled"] = True
        else:
            op_1_val = result_mem

    if ForwardingUnit.is_raw(op_2_reg, reg_dst_mem, control_unit.get_mem_wb_control()):
        raw_case_mem["operands"].append(2)

        if 2 in raw_case_ex["operands"]:
            raw_case_mem["forwarding_disabled"] = True
        else:
            op_2_val = result_mem
```

Figura 10: Llamadas de ForwardingUnit

2.2.5.1 Forwarding Unit

Es la clase responsable de detectar los riesgos RAW (Read After Write), cuya labor realiza a partir de una única función: `is_raw(op_reg, forward_reg_dst, forward_control)`. Este método se llama varias veces en la Figura 10 desde `instruction_decode()`, función a la que se le pasa como parámetros todos los datos necesarios para detectar el RAW, siendo:

- `result_ex`: Dato obtenido en EX
- `reg_dst_ex`: Registro destino de la instrucción en EX
- `result_mem`: Dato obtenido en MEM
- `reg_dst_mem`: Registro destino de la instrucción en MEM

Por cada operando e instrucción anterior en EX o en MEM se realiza una llamada pasando los atributos correspondientes:

- `op_reg`: Corresponde al registro desde donde se ha cargado uno de los operandos de la instrucción que se encuentra en ID.
- `forward_reg_dst`: Registro destino que se compara con el registro pasado en `op_reg`. En el caso de que coincida, también deberá asegurarse que la instrucción de donde procede el registro destino vaya a realizar una operación de escritura en el banco de registros.
- `forward_control`: Diccionario de señales de control correspondiente a la fase de segmentación de la instrucción con la que se está comprobando el RAW, donde en este caso se comprueba si la señal RegWrite está activa.

El motivo por el que se ha decidido modularizar tanto este método, hasta el punto de que hay que llamarlo varias veces por cada operando o registro destino en fases EX y MEM, es por facilitar

la identificación tanto del operando como del registro destino, en el caso de que haya un RAW. Si el ForwardingUnit detecta un RAW, en función de la llamada de `op_reg` que haya devuelto True, se sustituye el valor del operando correspondiente por el valor extraído desde la fase que se ha detectado el hazard, y se escribe en el registro de acoplamiento ID/EX. Además, en la implementación se tiene en cuenta la posibilidad de un posible doble RAW, donde la instrucción consumidora en ID puede depender de dos instrucciones productoras, una en EX y otra en MEM, que actualicen el mismo registro destino. En tal caso, se deberá priorizar la instrucción productora que se encuentra en EX, debido a que contiene el dato más actualizado. Para ello, empleamos el diccionario `raw_case_mem`, cuyo propósito principal es recabar los datos necesarios para poder mostrar correctamente el RAW en Terminal, al igual que `raw_case_ex`, pero se le ha incluido una entrada "forwarding_disabled", que se marca como True en caso de que compartan registro EX y MEM y se detecte un RAW en ambas fases, siendo el booleano que indica mediante un `if` si tiene lugar el forwarding respecto a la fase MEM

Se planteó la posibilidad de simplemente sobrescribir los datos transferidos desde MEM por los de EX simplemente extrayendo primero el dato de MEM y luego de EX, pero buscando una mayor fidelidad al hardware se decidió directamente detectar este caso e inhabilitar el forwarding de MEM.

2.2.5.2 Hazard Detection Unit

Esta clase permite identificar cuándo tiene lugar un riesgo de datos RAW, donde la instrucción productora del dato es un load, y además se encuentra en la fase EX. Debido a que esta instrucción debe estar en MEM para poder extraer el dato cargado desde memoria, es necesario aplicar un stall en IF e ID, para que la instrucción que se encuentra en decodificación espere un ciclo para así cargarle el dato, además de evitar que entren más instrucciones al cauce. Para detectar este hazard en específico, se implementa el método `is_load_raw(op_1_reg, op_2_reg, ex_reg_dst, ex_control)`, donde:

- `op_1_reg`: Primer operando de la instrucción en ID
- `op_2_reg`: Segundo operando de la instrucción en ID
- `ex_reg_dst`: Registro destino de la instrucción en EX
- `ex_control`: Conjunto de señales de control de la instrucción en EX

En este método se compara cada uno de los operandos de la instrucción en ID con el registro destino de la instrucción en EX, además de que se comprueban sus señales de control RegWrite y MemRead para verificar que es una instrucción load. A diferencia de ForwardingUnit, aquí no necesitamos identificar cada operando, o la fase de donde procede la instrucción productora (que en este caso solo puede ser EX), simplemente necesitamos saber si existe o no este riesgo, y en el caso de que esté presente, se aplican las medidas correspondientes. A diferencia de otros RAW, en este caso no es posible resolverlo en el mismo ciclo que se detecta, debido a que el dato que requiere la instrucción consumidora aún no ha sido cargado en el flujo, por lo que se introduce una burbuja en EX llamando a `control_unit.flush_id_ex()` desde la función de ID y vaciando el registro de acoplamiento ID/EX mediante `set_empty(True)`. Además, se devuelve como valor de retorno de la función de ID el valor True, que representa la señal Stall que se pasa como parámetro a la función de IF, donde para el cauce en las fases IF e ID.

2.2.5.3 Flush, Stalls y burbujas

En ocasiones, para poder solventar los distintos riesgos de datos y de control que se pueden presentar durante la ejecución del programa, es necesario modificar el cauce del camino de datos mediante flush, burbujas y stalls:

- `flush`: Se aplica a uno o varios registros de acoplamiento, inhabilitando la instrucción y sus señales de control.

- `stalls`: Inhabilita la sobrescritura de uno o varios registros de acoplamiento, paralizando la instrucción que contengan, retrasando uno o varios ciclos su ejecución.

- `burbujas`: Etapas de segmentación que no ejecutan ninguna instrucción válida.

Todos estos conceptos han ido apareciendo a lo largo de todo el documento, especialmente en secciones relacionadas con el tratamiento de riesgos, ya que son los mecanismos principales empleados para poder resolver riesgos que no se pueden solucionar mediante forwarding.

Un caso donde se emplean estos mecanismos es en los riesgos de datos RAW de tipo load donde la instrucción consumidora que está en ID debe esperar un ciclo para que la instrucción productora avance en el cauce y pueda obtener el dato actualizado desde memoria, y así poder transmitírselo a la instrucción consumidora. Ya se explicó anteriormente el proceso, pero si se detecta el RAW mediante Hazard Detection Unit, se aplica un stall tanto en el registro IF/ID como en el PC. Para ello, se devuelve un booleano correspondiente con el `stall_signal` que se pasa como parámetro a la función de IF e indica que se aplique el stall en caso de que su valor sea True.

También se aplican para inhabilitar instrucciones posteriores a una instrucción de salto o bifurcación, que han entrado al cauce del camino de datos debido a que son las instrucciones posteriores a esta instrucción, y que el salto no se ejecuta hasta la fase EX, pudiendo entrar instrucciones que no corresponden al flujo. Para identificar si el salto se ejecuta en EX, la función `execute()` devuelve una tupla de valores, siendo uno de ellos jump_address, que contiene la dirección de salto si se ha realizado, o None en caso contrario. Esta variable se comprueba en la Figura 8 mediante un `if`, y si el valor de la variable no es None, entonces el salto se ha realizado, y por tanto hay que aplicar flush en ID/EX y establecerlo como registro vacío, generando una burbuja mediante `set_empty(True)` y `flush_id_ex`. En el caso de `instruction_fetch()`, simplemente se pasará la variable `pc_src`, que indica si `jump_address` no es None, y el propio `jump_address`, cargando en IF la instrucción correspondiente al salto ejecutado.

2.3. Simulación y salida

En esta sección se pondrá a prueba nuestra simulación ejecutando un programa en ensamblador completo, con todo tipo de instrucciones y riesgos de datos y control. En primer lugar se explicará cómo representamos la ejecución de nuestra simulación en terminal, explicando la información que se muestra e indicando los datos y variables requeridos para su implementación. Después se presentará el programa a ejecutar, explicando brevemente las instrucciones que contiene y se hará una comparación entre los resultados esperados y los resultados obtenidos. Finalmente se hará un análisis de casos más concretos dentro del programa, haciendo especial enfoque a los riesgos de datos y control.

2.3.1. Representación de los ciclos

Para representar cada ciclo del programa se emplea el método `print_execution(cycle)`, que pertenece a la clase ProcessingUnit, facilitando el acceso tanto a los registros de acoplamiento como a los componentes (memorias, ALU, banco de registros y PC) para poder representar el flujo. Además, se le pasa como parámetro el ciclo de ejecución en el que se encuentra, el cual se obtiene en el método `run()` de CPU. Para poder mostrar las operaciones realizadas en cada ciclo, además de los posibles RAW, se implementan como atributos de la clase ProcessingUnit:

- `operations`: Un diccionario que contiene una descripción de la operación realizada en cada fase de segmentación.

- `raw_list`: Lista que contiene los diccionarios `raw_case_ex` y `raw_case_mem`, diccionarios que almacenan información relativa al riesgo de dato que haya tenido lugar, ya sea con la

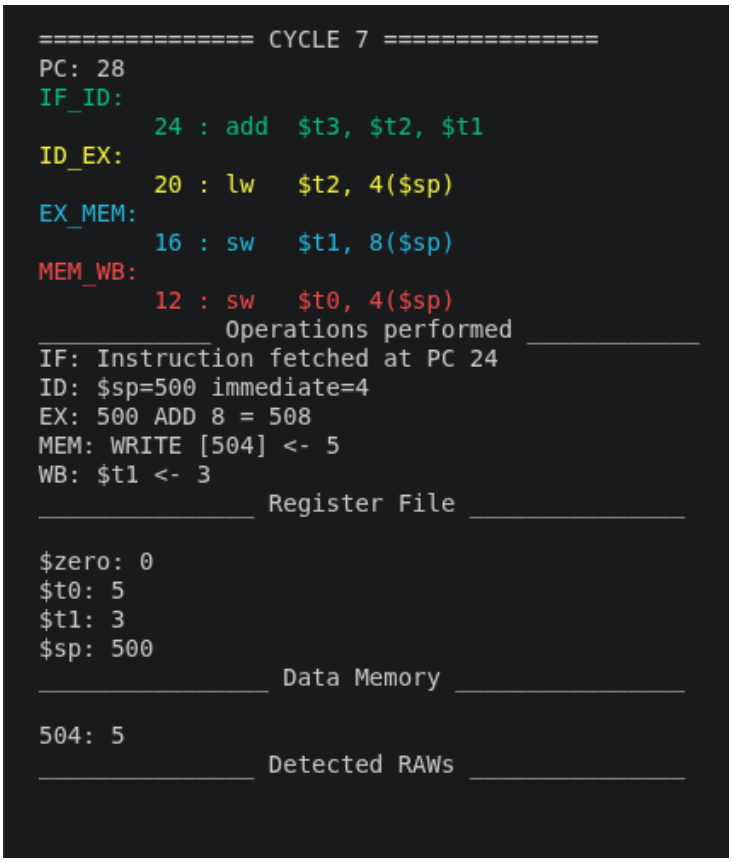


Figura 11: Representación de ciclo.

instrucción productora en EX, MEM o ambas, para poder representarlo correctamente por terminal.

También, en los registros de acoplamiento se emplea el atributo `instr_pc`, orientado únicamente para mostrar el PC de la instrucción que contienen, en el caso de que no estén vacíos.

El resultado de nuestra representación de cada ciclo se refleja en la Figura 11. Respecto a la información que se muestra en cada ciclo, podemos dividirla en cinco franjas:

- PC y Registros de acoplamiento: Se muestra el valor actual de PC, además de las instrucciones y los PCs correspondientes almacenados en cada registro de segmentación. Además, los registros de acoplamiento IF_ID, ID_EX, EX_MEM, MEM_WB se muestran con los colores verde, amarillo, azul y rojo respectivamente.

- Operaciones realizadas: Se muestra una breve descripción de las operaciones realizadas en cada fase de segmentación, las cuales se extraen de `operations`.

- Banco de registros: Muestra todos los registros del banco de registros que tengan algún valor que no sea None, que inicialmente son `$zero` y `$sp`.

- Memoria de datos: Refleja todos los datos que hayan sido escritos en la memoria de datos, que inicialmente se encuentra vacía.

- RAW detectados: En el caso de que se detecte algún riesgo de datos, se representa la información relativa al riesgo extrayéndola de `raw_list`.

```
1   .text
2   main:
3
4       # Inicialización
5       addi $sp, $sp, -12
6       addi $t0, $zero, 5
7       addi $t1, $zero, 3
8
9       # Escribir en memoria
10      sw   $t0, 4($sp)     # RAW t0 (falta 1 instr)
11      sw   $t1, 8($sp) # RAW t1 (falta 1 instr)
12
13      # Cargar desde memoria
14      lw   $t2, 4($sp)
15      add  $t3, $t2, $t1  # RAW $t2 (MEM)
16
17      # Operaciones aritmético-lógicas (RAWs varios)
18      slt  $s1, $t1, $t0      # 3 < 5 = 1 -> s1
19      slti $s2, $t0, 4        # 5 < 4 = 0 -> s2
20      and  $t4, $s1, $s2      # 1 and 0 = 0
21      or   $t5, $s1, $s2      # 1 or 0 = 1
22      xor  $s0, $t4, $t5      # 0 xor 1 = 1 -> s0
23      add  $s3, $t0, $t1      # 5 + 3 = 8 -> s3
24      sub  $s4, $s3, $t1      # RAW en $s3 (EX)
25
26      # Salto condicional forward (true)
27      beq  $s0, $s1, forward
28
29      # Instruccion se cuela y debe invalidarse
30      addi $t6, $zero, 6
31
32  forward:
33
34      # Bucle de 2 iteraciones
35      addi $s6, $zero, 2
36  loop:
37      addi $s6, $s6, -1
38      bne  $s6, $zero, loop     # salto backward
39
40      # Salto incondicional forward
41      j    end
42
43      # Debe invalidarse por el j
44      addi $t7, $zero, 7
45
46  end:
47      sw   $s0, 12($sp)
48      add  $v0, $s0, $zero
```

Figura 12: Programa principal de prueba en ensamblador MIPS32.

2.3.2. Programa principal de prueba

El programa principal se presenta en la Figura 12, y es un programa que contiene, sin tener en cuenta los bucles: 16 operaciones aritmético-lógicas, de las cuales 8 son con inmediato; 4 accesos

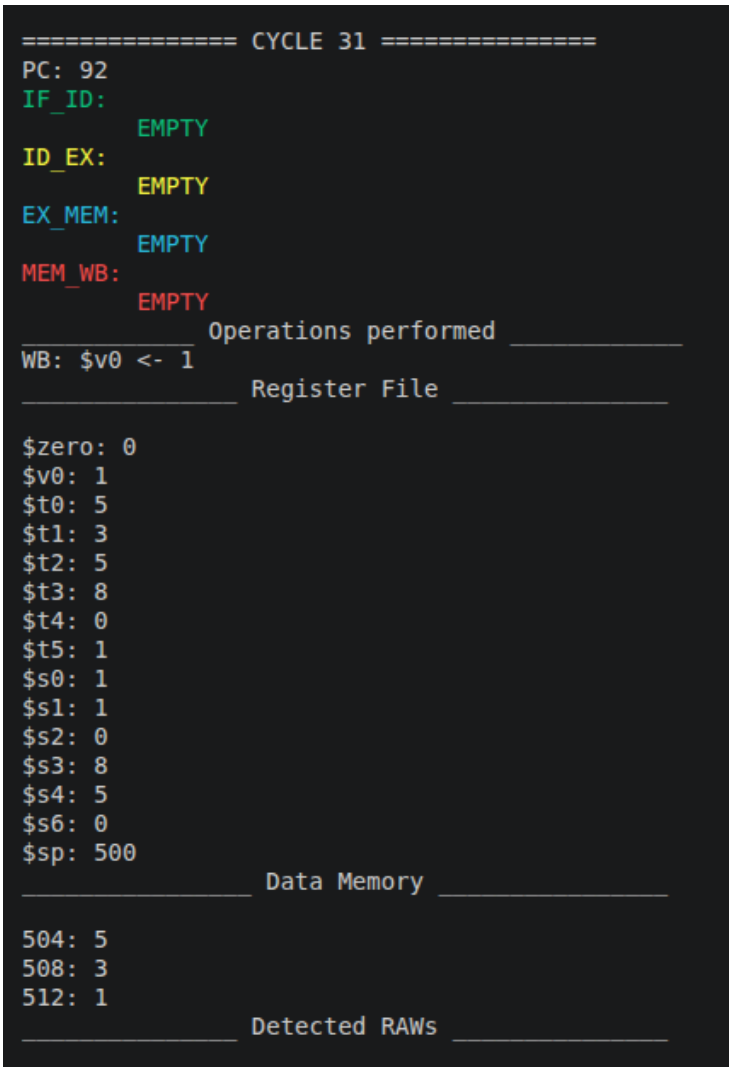


Figura 13: Resultados de la ejecución.

a memoria, 3 mediante store y 1 con load; y 3 saltos, 2 de ellos condicionales. La ejecución del programa se ha realizado en 31 ciclos, y los resultados obtenidos se reflejan en la Figura 13. Los datos almacenados en la memoria de datos corresponden a las escrituras con desplazamiento realizadas mediante la dirección almacenada en el registro `$sp`, siendo 504 ← 5, 508 ← 3, 512 ← 1. Respecto a los valores de los registros, como referencia se calculó el resultado en los registros, siendo: `$sp` ← 500, `$t3` ← 8, `$s1` ← 1, `$s2` ← 0, `$s6` ← 0, `$v0` ← 1. Todos ellos coinciden con los resultados obtenidos.

2.3.3. Casos específicos

Tras una comprobación general de los resultados obtenidos después de ejecutar completamente el programa en ensamblador se procede a analizar casos concretos de la ejecución, especificamente, se analizarán distintas situaciones de riesgos de datos y riesgos de control, comentando tanto la detección como resolución de cada uno de ellos.

2.3.3.1 RAW convencional

Durante el ciclo 12 tenemos en la fase de ID la instrucción `and $t4, $s1, $s2`, pero en la sección de RAW detectados en terminal observamos que tenemos dos RAW, uno por cada operando: `slti $s2, $t0, 4` en EX y `slt $s1, $t1, $t0` en MEM. Sin embargo, en la Figu-

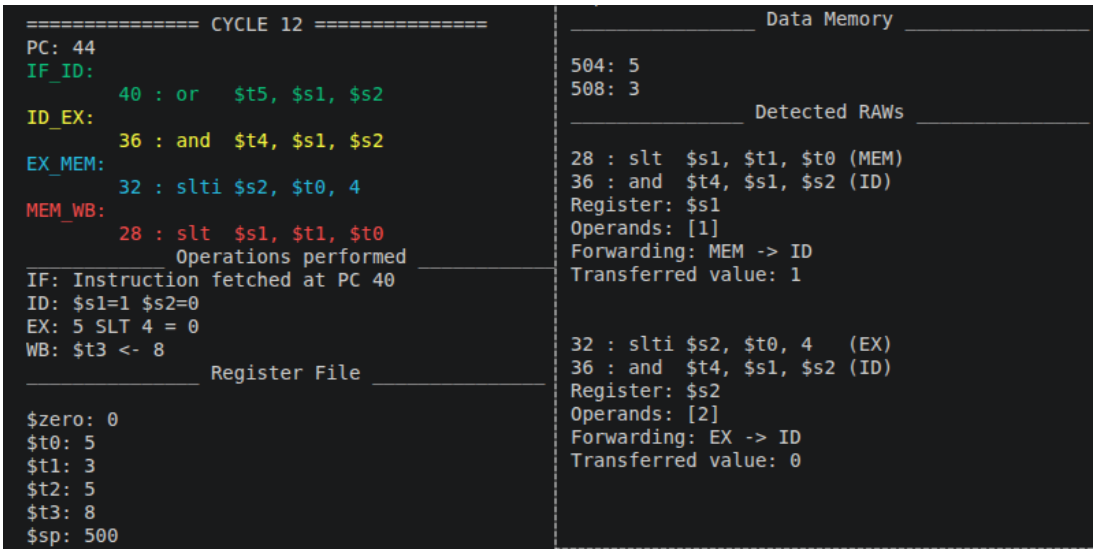


Figura 14: RAW convencional

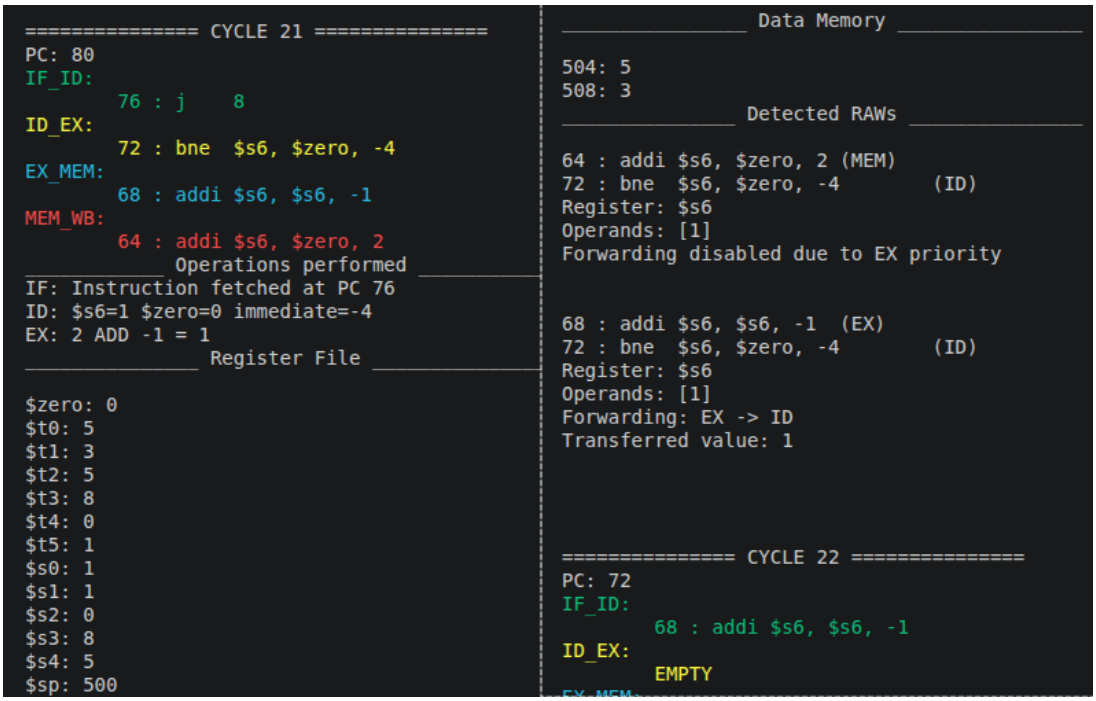


Figura 15: RAW simultáneo desde EX y MEM.

ra 14 observamos que se realiza con éxito el forwarding de ambos valores sin ningún problema, transfiriendo el valor 0 del registro `$s2` y el valor 1 del registro `$s1`.

2.3.3.2 RAW simultáneo desde EX y MEM

En este caso también tenemos dos RAW desde distintas fases, pero ahora ambos riesgos de datos son respecto al mismo registro operando de la instrucción en ID. Como vimos anteriormente, la implementación del forwarding está diseñada de tal manera que se priorice el dato procedente de la fase EX, debido a que corresponde con el dato más actualizado de los dos. En la Figura 15 se observa el mensaje "Forwarding disabled due to EX priority" en la sección de Detected RAW, lo que nos indica que el dato transmitido a la instrucción en ID ha sido el que se ha calculado en EX.

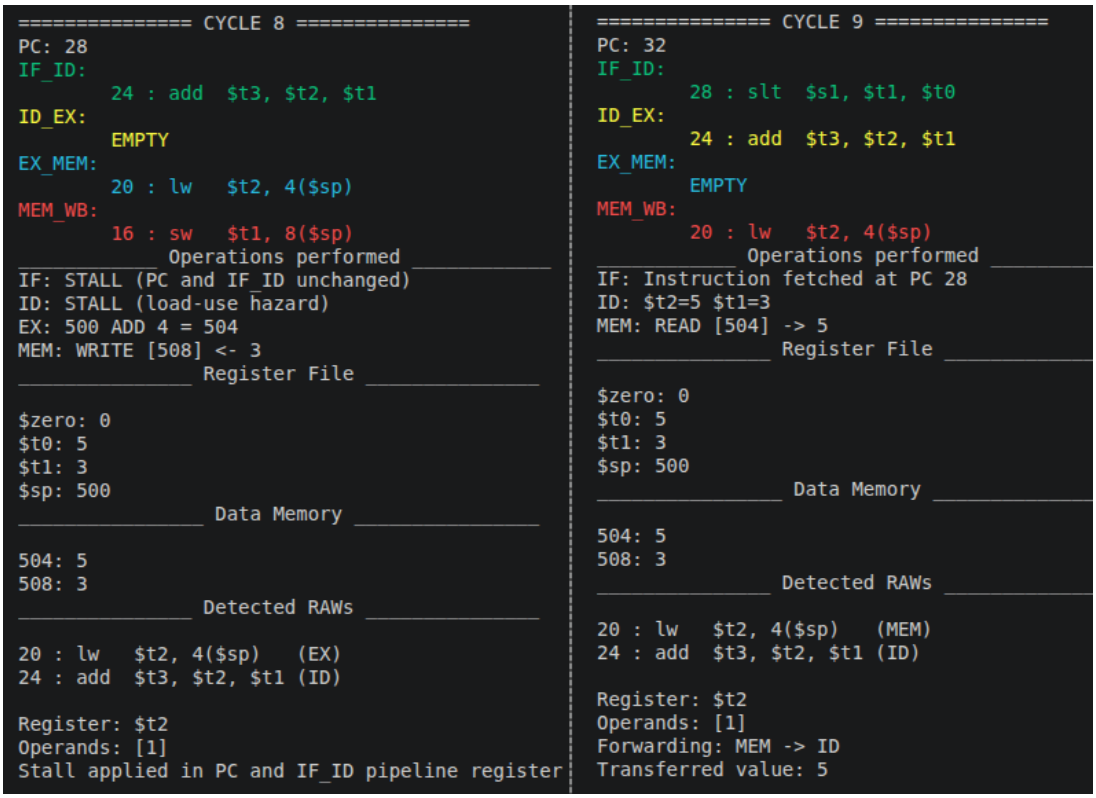


Figura 16: RAW con load desde EX.

2.3.3.3 RAW con load desde EX

En este último riesgo de datos nos encontramos con uno que no se puede resolver mediante un forwarding inmediato en la fase EX, y es que hay presencia de un RAW respecto a una instrucción `add` en fase de decodificación y la instrucción de tipo load que se encuentra en la fase de ejecución. En RAW donde la instrucción productora es un load en fase EX, no podemos anticipar el dato debido a que aún no lo hemos cargado desde la memoria de datos. La solución es paralizar el cauce en PC y en el registro de acoplamiento IF/ID, para que avance la instrucción load y poder anticipar el dato desde la fase MEM, siendo el 5 leído de la dirección 504 en la memoria de datos. En la Figura 16 se muestran los ciclos 8 y 9, donde primero, en el ciclo 8, se aplica el stall en PC e IF/ID (como se observa en Operations performed), introduciendo una burbuja en ID/EX, y por tanto en EX en el siguiente ciclo. Además, en la sección de Detected RAW nos indica que se ha detectado un riesgo al que se ha tenido que aplicar stall. Gracias a estas medidas, en el ciclo 9 observamos en Detected RAW cómo la instrucción `add` ha recibido el dato procedente de la instrucción load desde la fase de memoria.

2.3.3.4 Riesgo de control tras ejecución de salto

Para este caso concreto tenemos un riesgo de control debido a la ejecución de un salto. En la Figura 17 observamos cómo en el ciclo 17 tenemos una instrucción de salto condicional en el registro de ID/EX y una instrucción arbitraria en el registro de IF/ID, por lo que, si se ejecuta el salto condicional de la instrucción `beq`, habrá que retirar del cauce la instrucción que se encuentra en IF/ID, ya que se encuentra fuera del flujo de ejecución. En el ciclo 18 podemos observar en "Operations performed" que el salto se ha ejecutado con éxito, y que además de cargar la instrucción correspondiente a la dirección del salto, también se ha inhabilitado la instrucción que no correspondía al flujo, creando una burbuja. También se muestra otra burbuja en EX/MEM debido a que la instrucción `beq` ha terminado su ejecución en EX, y no tiene ninguna otra operación pendiente que ejecutar en las etapas posteriores. Cabe recalcar que, si el salto hubiera

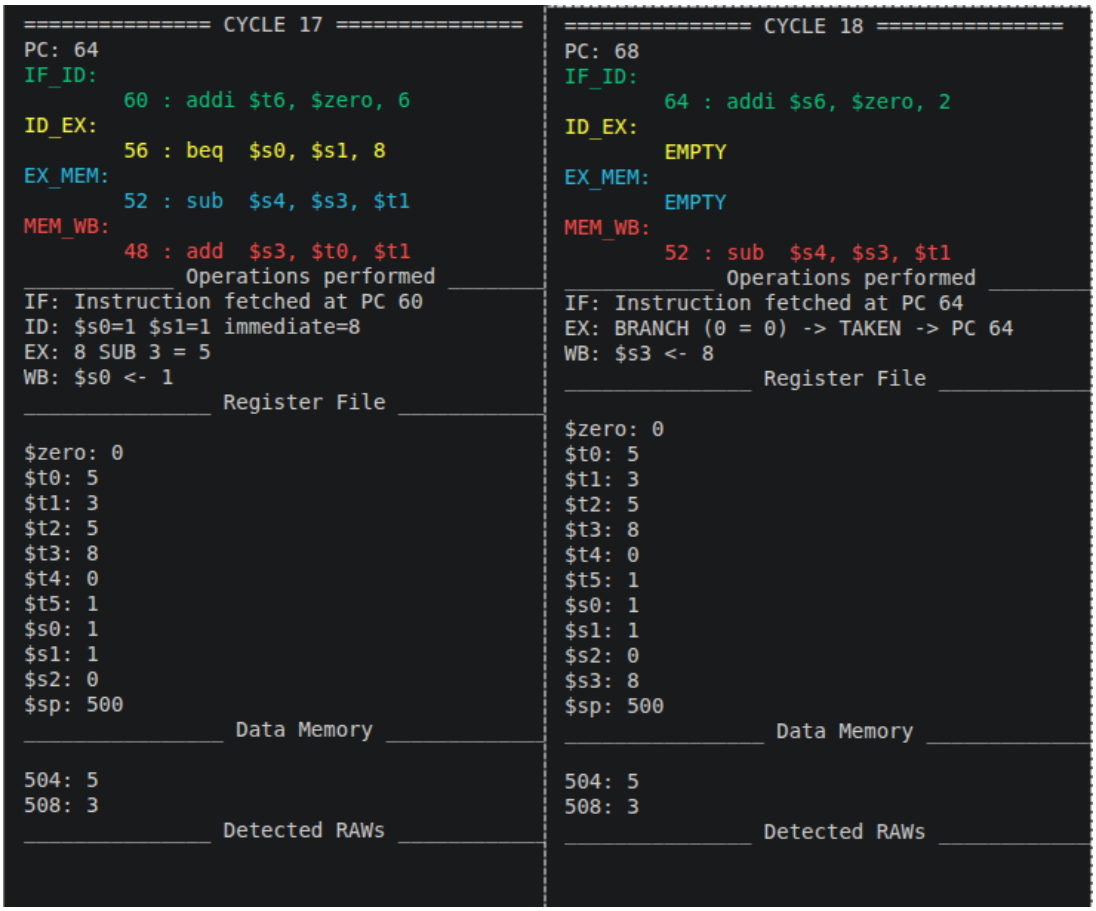


Figura 17: Riesgo de control tras ejecución de salto.

sido incondicional, el tratamiento del riesgo de control sería el equivalente, aplicando el mismo método para resolver el riesgo de control.

3. Conclusiones

Retrocediendo a la introducción de la práctica, el objetivo principal era implementar una simulación en Python de un procesador MIPS32 segmentado, objetivo que se ha logrado mediante el fraccionamiento de las instrucciones en cinco fases de segmentación. Para la correcta ejecución de cada fase de segmentación se han implementado los distintos componentes requeridos en nuestro camino de datos, siendo las memorias de instrucciones y datos, un banco de registros, una unidad aritmético-lógica y un contador de programa, además de los distintos registros de acoplamiento. Nuestra simulación es capaz de ejecutar instrucciones aritmético-lógicas, ya sean con o sin inmediato, operaciones de acceso a memorias de load y store, y saltos condicionales e incondicionales. Además, se han implementado los mecanismos de forwarding y de detección de hazards correctamente, permitiendo resolver los distintos riesgos de datos y de control que puedan presentarse en nuestro flujo de ejecución.

Gracias al diseño orientado a objetos se han podido abstraer y separar las distintas responsabilidades que cada componente o mecanismo del sistema tenía asignadas, pero a la vez creando una cohesión que ha facilitado el acceso a distintos atributos y métodos, permitiendo crear implementaciones más eficientes. La principal división de responsabilidades se ha realizado entre las clases ProcessingUnit y ControlUnit, ya que la unidad de procesamiento contenía todo el hardware, mientras que la unidad de control era la responsable de transmitir cada una de las señales de control de cada instrucción en el camino de datos. Además, esta alta cohesión y bajo acoplamiento producto del diseño orientado a objetos otorga al código una mayor facilidad de comprensión y de mantenibilidad.

El diseño e implementación de un procesador con cauce segmentado conlleva enfrentar todo tipo de dificultades tanto de diseño como de implementación. El primer reto ha sido el propio diseño, donde se han tenido que tomar decisiones relacionadas con la aplicación de herencia a clases asociadas con las memorias o y su posible aplicación en los registros de acoplamiento, la separación de responsabilidades respecto a las unidades de procesamiento y control, la forma correcta de representar datos como pueden ser instrucciones o direcciones de memoria, y todo esto tratando de mantener siempre una fidelidad al hardware del camino de datos, pero creando un software que fuera eficiente. Después, durante la implementación, también hubo muchas dificultades en distintas fases del desarrollo del código. Durante la creación de la función encargada de la lectura del programa en ensamblador, hubo bastantes problemas iniciales respecto al tratamiento de labels y las direcciones de salto de algunas instrucciones. También, en la implementación del cauce segmentado de instrucciones, costó mucho encontrar una solución a la sobrescritura de EX/MEM, debido a que había que ejecutar la fase de EX antes que la de MEM sin que se perdiera la información del registro EX/MEM que la fase MEM requería. Además, realmente no se pudo poner a prueba la implementación de la simulación hasta ya una fase mucho más avanzada del desarrollo, donde ya era funcional y procesaba código, pero debido a que hasta entonces no habíamos podido ejecutar código, se habían ido acumulando distintos bugs que requirieron bastante tiempo para ser arreglados. Después, la implementación del ForwardingUnit y del HazardDetectionUnit, pese a que sus métodos de detección de riesgos de datos no suponieron ningún problema, implementar toda la lógica de señales flush y stalls en función del tipo de riesgo de dato y la fase en la que se había detectado también fue un reto.

Al final, se ha logrado una simulación segmentada, que puede procesar un programa en ensamblador básico que esté conformado por operaciones básicas de cálculo, acceso a memoria y de salto, del cual se puede hacer un seguimiento detallado de cada uno de los ciclos. Se ha puesto a prueba la simulación, no solamente con el programa de prueba establecido previamente, sino también con casos más concretos relacionados con los riesgos de datos y de control que son necesarios cubrir, pero no se ha llegado a implementar programas en ensamblador que pongan a prueba situaciones aún más concretas, por lo que el programa probablemente contenga algún posible bug, además de algunos que se han obviado como la ejecución de un programa en

ensamblador vacío, o que contenga instrucciones que no estén incluidas en la implementación de nuestro procesador.

A modo de conclusión, esta práctica, pese a que ha sido realmente complicada en ciertos puntos, me ha servido para entender e interiorizar todo el proceso requerido en un procesador segmentado, incluyendo la interacción entre los distintos elementos del procesador, ya sea desde una capa lógica o de procesamiento, la coordinación de distintas instrucciones segmentadas y la transmisión de datos entre distintos elementos de la simulación.

Bibliografía

Patterson, D. A., & Hennessy, J. L. (2000). *Estructura y diseño de computadores* (Vol. 2). Reverté.